

Using Epistemic Observability in Agentic Systems for Combating Hallucinations

Tony Mason

The University of British Columbia

Vaastav Anand

Max Planck Institute for Software Systems

Abstract

Systems incorporating language model components face a verification problem: these components fabricate, and verification has cost. How should a system allocate its verification budget? Systems engineers building with language model components need to know what each level of verification investment buys. This paper provides the formal framework and empirical cost surface for making that decision.

We prove that text-only observation with bounded supervision is structurally insufficient to verify epistemic honesty, regardless of model scale or training procedure (the impossibility is architectural, not a capability gap). To overcome this, we introduce epistemic observability and construct a tensor interface that escapes this impossibility by exporting internal computational signals, such as entropy traces and attention summaries, that the model cannot independently control under current training objectives. Under a length-controlled evaluation where text-channel features collapse, per-token entropy remains discriminative, and tensor-guided judges outperform text baselines at every verification budget level (+2.5–3.9 percentage points, averaged across four architectures). Moreover, we show that tensor-guided judges outperform text baselines with lower verification budgets.

1 Introduction

Systems incorporating language model components face a verification problem. The components generate text by sampling from learned probability distributions, and some fraction of that text is fabricated: fluent, confident, and wrong. LLM-based systems today are rife with “hallucinations”.

We use the term *fabrication* throughout this paper to emphasize that the system’s default behavior under coherence optimization is reasonable and not a malfunction to be patched. Language models optimized for coherence produce coherent output. The output reflects the training data: grounded training data tends to track truth, while ungrounded training data tends to produce untruth. Xu et al. [14] proves that hallucination is inevitable for LLMs over the class of computable functions.

Consequently, every system that deploys a language model component must allocate a budget: how much of the output to verify, which outputs to prioritize, and what signals to use for triage. For example, checking a citation against a database costs API calls and latency; having a domain expert review a summary costs time and salary;

running a second model as a judge costs compute. Verification cost varies by content type: checking a citation is cheap; assessing whether a summary faithfully represents its source is expensive.

The key issue is that the current text-only interface to language models does not expose the model’s internal computation, which produces signals that distinguish grounded generation from fabrication. The model may know it is fabricating but currently the supervisor cannot tell if the model is fabricating just from the text output.

This observability gap has direct consequences for verification cost. A supervisor receiving only text must spend its verification budget reconstructing, from the output alone, what the model’s own computation already knew. The budget is consumed by reconstruction rather than concentrated on the cases where verification is most needed.

Our key insight is that language models carry internal signals, which are the entropy traces, attention geometry, token-level log-probabilities, that distinguish grounded generation from fabrication. These signals are byproducts of the computation itself: *telemetry* that the model cannot separately control under standard training. Unlike the model’s *testimony* (the text it chooses to output), telemetry is harder to fake.

We introduce *epistemic observability*: an interface that exposes this telemetry alongside the standard text output. Unlike interpretability, which seeks to explain why a model activates particular neurons or circuits, or explainability, which generates post-hoc rationales for its outputs, epistemic observability addresses a different question: how trustworthy is this generation? Rather than explaining the model’s internal reasoning, it surfaces signals produced as a byproduct of inference that help estimate whether the generated content is grounded in the model’s knowledge or is likely to be fabricated.

To support a range of application requirements, we propose a multi-tier epistemic observability interface, where each successive tier exposes richer and more informative telemetry about the generation process. These tiers provide a principled trade-off between observability and computational cost, enabling developers to select the level of epistemic insight that best matches their system’s reliability requirements and resource budget. For instance, a creative brainstorming assistant may require only Tier 0 observability, whereas a medical decision-support system may justify the additional cost of higher observability tiers to better identify and validate potentially hallucinated content. In this way,

Type	Description
Adversarial Truth	True but implausible (“combat wombat is an Australian military mascot”)
Shattered Lie	No training-data grounding (“Treaty of Westphalia II”)
Deceived Lie	Pattern-matches plausible language (“Glavinsky syndrome”)
Confused Truth	Counterintuitive fact (“more camels in Australia than Egypt”)

Table 1. Hallucination Types that span the verification space.

the observability hierarchy makes explicit what additional epistemic information each level of investment provides.

As a proof of concept, we build a new tensor interface at Tier 1 epistemic observability, which includes the entropy at the token granularity. [TODO: 1 liner about the eval](#)

2 Background & Motivation

A language model maps input tokens to output tokens, producing internal signals (entropy, attention patterns, logprobabilities) as computational byproducts. The training corpus contains both truth-tracking institutional text and misinformation. This results in four canonical hallucination types, shown in [Table 1](#).

A language model trained on this corpus inherits these mixed regularities. Base models (pre-fine-tuning) fabricate substantially on factual benchmarks because they are not honest by default. But they also encode rich internal representations of concepts like correctness and truthfulness, which can be elicited under the right conditions [3]. The base model’s relationship to truth is complex: neither reliably honest nor uniformly deceptive. The question for systems engineers is not whether the model is honest, but what it costs to verify when it is not.

2.1 Testimony & Telemetry

Architecture primer. A transformer-based language model generates text one *token* (subword unit) at a time, autoregressively: at each step it produces a probability distribution over its vocabulary, selects a token, and conditions the next step on all prior output. Each step passes through a stack of layers producing *attention patterns* (weighted relevance scores between positions) and intermediate representations. The *entropy* of the output distribution, the attention patterns, and the *log-probabilities* of selected tokens are byproducts of this computation: telemetry the model cannot control.

Existing models operate with a text-only output interface. The text is the testimony, whereas the telemetry are measurements of the computation that produced the output. The model can control its testimony; under current training objectives, it cannot independently control its telemetry. From a cost perspective: testimony is cheap to produce and

cheap to inspect but unreliable; telemetry is more expensive to export and process but, under current training regimes, harder to game because it is a byproduct of the computation rather than a product of the model’s optimization objective. This decoupling is an empirical property of existing training pipelines, not an architectural guarantee: whether adversarial training could learn to couple telemetry to the optimization objective is an open question (??).

2.2 Coherence over Factuality

Existing models today optimize for generating coherent output instead of factual output [5]. Under these conditions, the optimization pressure favors fluent completion over honest refusal. This can be formalized as a reward inequality, as shown in [Figure 1](#).

$$w_{\text{coherence}}R_{\text{coherence}} + w_{\text{engagement}}R_{\text{engagement}} \\ \gg w_{\text{factuality}}R_{\text{factuality}} + w_{\text{refusal}}R_{\text{refusal}}$$

Figure 1. Reward inequality for favoring fluent completion over honest refusal

No direct measurement of the weights from [Figure 1](#) in commercial reward models (separate models trained on human preferences that score language model outputs) exists. But current evaluation and feedback practices incentivize behavior consistent with such an inequality. The False-Correction Loop emerges: models apologize, claim correction, and fabricate again, because conversational fluency dominates factual accuracy in the effective reward signal.

Reinforcement Learning from Human Feedback (RLHF) [10] *can* teach models to recognize knowledge boundaries when the reward signal is carefully constructed. TruthRL [13], which uses ternary rewards distinguishing “correct,” “hallucinated,” and “abstain,” demonstrates 28.9% hallucination reduction and 21.1% truthfulness improvement on knowledge-intensive benchmarks.

However, RLHF optimizes for text-channel reward, which are precisely the channel that our impossibility theorem (??) shows is insufficient for verification. To our knowledge, public descriptions of major RLHF pipelines rarely describe ternary rewards or explicit abstention incentives. Documented RLHF practice often conflates helpfulness, harmlessness, and honesty under vague “HHH” criteria [2], with definitional slippage obscuring trade-offs between user-friendliness and truthfulness. Current popular benchmarks and leaderboards give zero or minimal credit for abstention, making confident guessing more score-optimal than calibrated uncertainty. The result is that RLHF tunes the behavioral surface (what the model says) without altering the computational substrate (how the model generates). This is the text-channel trap: the optimization pressure operates on the gameable signal.

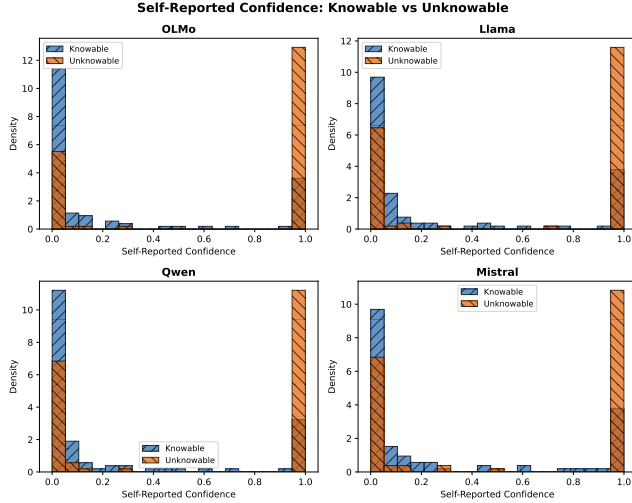


Figure 2. Self-reported confidence is inverted: all four models report higher confidence on unknowable queries (AUC 0.28–0.36, below random), while tensor signals (entropy, AUC 0.89–0.95) correctly discriminate knowable from unknowable. The text channel measures the model’s behavioral response to an elicitation prompt; the tensor channel measures the computation alone.

2.3 The Text Channel Cannot Self-Verify

Current models cannot be trusted to honestly report their own epistemic state through the text channel. To demonstrate the inadequacy of text-only verification, we use a taxonomy of query types: *knowable* queries (facts the model should have stable representations for) versus *unknowable* queries (fabrication prompts, future events, private information) and report each model’s self-confidence score for three text-channel baselines, which are the cheapest verification strategies available to a bounded supervisor:

- **Baseline 1: Self-Reported Confidence.** Ask the model “How confident are you?” after generation. Tests whether the model can introspect and report its epistemic state through text.
- **Baseline 2: Hedging Language.** Count hedging markers (“I think,” “possibly”) and refusal markers (“I don’t know,” “cannot verify”) in the response.
- **Baseline 3: Response Length.** Use output length as an uncertainty proxy: uncertain responses may be unusually short (refusal) or long (padding).

The self-report baseline is particularly instructive. Self-reported confidence is *inverted*: all four models report higher confidence on unknowable queries than on knowable ones, yielding AUC 0.28–0.36 across architectures (all below random). Tensor-based discrimination is both correctly oriented and stable (AUC 0.89–0.95) across the same four architectures. A naive user who asks “how confident are you?” receives a number that anticorrelates with actual epistemic grounding, while an adversary could exploit this inversion to make fabricated outputs appear maximally confident.

By contrast, the tensor channel is a function of the model’s computation alone. Entropy is not elicited; it is measured. The prompt format does not affect the discriminative power of the measurement because the measurement is a byproduct of generation, not a response to a question about generation. This asymmetry, testimony depends on the elicitation method, telemetry does not, is the empirical core of the interface problem.

Under realistic mixed-objective incentives and bounded oversight, the text-only interface is insufficient: a supervisor cannot distinguish truthful uncertainty from confident fabrication. The next section formalizes this observation gap.

3 Epistemic Observability

The formal impossibility establishes a budget floor: below the tensor interface, verification investment in the text channel yields diminishing and gameable returns. The premise is that the internal state of a language model carries epistemic information not exported through the text-only interface. In systems terms, the model is *unobservable*: it maintains internal state that affects output correctness but is invisible to external consumers.

The work ahead is building AI systems that export their epistemic state the way distributed systems learned to export their causal state. Not as an afterthought. Not as an optional feature. As a fundamental design requirement. The tensor interface increases the observation budget available to external verifiers.

Concretely, we define *epistemic observability* to include not just text, but structured metadata, such as answer status (answer, abstain, or uncertain), provenance (retrieved sources or explicit “no grounding”), alternative hypotheses the system considered, and falsifiers (what evidence would change the conclusion). Such metadata is harder to fabricate than fluent text, especially when coupled to verifiable external state. The design space is large; the constraint our theorems impose is that *some* epistemic export is required because the text-only interface provably cannot suffice.

3.1 Principles of Epistemic Observability

The impossibility results in ?? apply to a specific architectural class: systems operating under text-only observation models with bounded supervision. They are, in essence, a consequence of unobservability: the interface discards the internal state that verification requires. They do not preclude epistemically honest AI systems in general. They characterize the conditions under which honesty guarantees become achievable, which is to say, the conditions under which the relevant internal state becomes observable. We identify three architectural properties of epistemic observability that, together, place a system outside the impossibility class:

Principle 1 (State Exteriority). *The representation of validity must be separated from the representation of generation. The*

policy must explicitly condition on retrieved world-state w , not just query q . This breaks the representational impossibility (??) by giving the system access to the information needed to distinguish answerable from unanswerable queries.

What counts as “external reality” matters. A web corpus is external to the model but not independently verified; it may contain the same fabrications the model would generate. State Exteriority requires grounding in sources with their own integrity guarantees: curated databases, sensor data, cryptographically signed records, or oracles with verifiable provenance.

Retrieval-augmented generation exemplifies partial progress by grounding generation in retrieved documents, but corpus co-occurrence is a proxy for truth, not truth itself. Retrieval-augmented systems still fabricate citations because State Exteriority is necessary but not sufficient.

Principle 2 (Verification Independence). *Verification signals must originate from a channel orthogonal to the generation signal, capable of overriding the bounded supervisor’s preference for plausibility. This breaks the learnability impossibility (??) by providing a training signal that distinguishes honest from fabricated responses.*

The model has no incentive to lie in the verification channel because doing so does not improve its task score. Full Verification Independence requires the verification channel to access internal state or external ground truth, not just a second text output.

Principle 3 (Provenance Binding). *Every output assertion must be structurally bound to a verifiable source. This addresses the composition problem: as responses grow in complexity, cross-claim consistency checking becomes superlinear. Provenance binding makes verification modular because each claim can be checked against its source independently.*

Current retrieval-augmented systems gesture toward this but rarely achieve it. Citations are generated, not bound. The model produces text that looks like it cites sources, but the citation is itself a generation subject to fabrication. True provenance binding requires structural guarantees: signed queries to retrieval services, content hashes of retrieved passages, or justification graphs that can be independently audited.

3.2 Epistemic Signals

We claim the tensor reveals whether the model is operating in *grounded mode* (retrieving from stable representations) versus *fabrication mode* (generating without anchoring).

We extract three per-token signals from the generation process:

1. **Entropy:** the Shannon entropy of the output probability distribution at each token position. High entropy indicates the model is uncertain among many candidates; low entropy indicates a confident next-token prediction.

2. **Top- k probability mass:** the cumulative probability of the five most likely tokens at each position. Low top- k mass indicates the model’s probability is spread across many alternatives.
3. **Mean log-probability:** the average log-probability of the tokens actually selected. Low values indicate the model chose low-probability tokens, suggesting generation without strong grounding.

These signals are byproducts of the softmax computation at each generation step. They require access to the output distribution, not to internal activations, making them extractable with minimal interface changes.

3.3 The Tensor Interface

We propose a minimal interface that demonstrates the escape is achievable with current architectures. The key insight is the distinction between *testimony* (what the model says) and *telemetry* (measurements of the computation that produced the output).

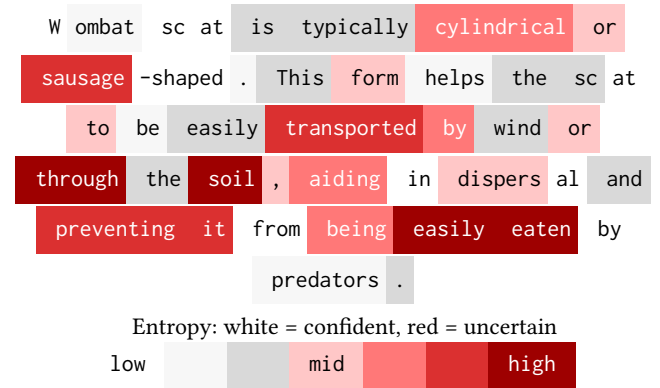


Figure 3. Generated output consisting of 43 tokens along with its entropy trace for the query *What shape is wombat scat?* with mean entropy $H = 1.147$, max entropy $H = 4.315$. Token background color indicates per-token entropy: white (confident/low entropy) through pink to red (uncertain/high entropy).

Definition 3.1 (Tensor Interface). A generation interface that returns:

1. **Text:** The linearized response string (standard interface output).
2. **Entropy trace:** Per-token entropy of the output distribution during generation.
3. **Attention summary:** Statistics of attention patterns (concentration, self-attention ratio) from deeper processing layers.

This interface exports telemetry that the model cannot separately control under current training objectives. The model can lie in its text, but it cannot present a different inference computation than the one it actually performed. Independent

```

1 def generate_with_tensor(prompt) -> (text,
    entropy_trace, attention_summary)

```

Figure 4. Epistemic Observability Tensor Interface

work on persona monitoring confirms this asymmetry: activation projections reveal that models adopt distinct internal configurations for different epistemic conditions even when producing textually similar outputs [7]. This asymmetry is the foundation of the adversarial robustness argument: under current training objectives, computational signals (entropy, attention geometry) cannot be independently controlled by the model without changing the generation process itself, unlike behavioral signals (response length, hedging patterns) that can be adapted through training without altering what the model knows or does not know. Figure 3 shows the generated output for the query *What shape is wombat scat?* along with its entropy trace. We color code per-token entropy: blue indicates low entropy (confident generation), orange indicates moderate entropy, and brown indicates high entropy (uncertain generation). The entropy enhanced output shows where in the output generation did the generation proceed with high confidence (low entropy, suggesting retrieval of stored pattern) and where did it proceed with high uncertainty (high entropy, suggesting fabrication).

Compositional defense. The tensor’s blind spots characterize where other verification methods are most effective. §3’s three principles — State Exteriority, Verification Independence, Provenance Binding — need not be satisfied by a single mechanism. The tensor provides Verification Independence: an orthogonal signal the model cannot separately control. Bounded judges provide State Exteriority: external lookup against verifiable sources. Input-side defenses (prompt analysis, supply chain detection) provide Provenance Binding at the system boundary. Each layer addresses the failure class where the others are weakest. The tensor catches uncertain fabrication where verification is expensive; the bounded judge catches confident fabrication where verification is cheap; input-side analysis catches adversarial inputs designed to exploit well-learned patterns. Composing these layers allocates a finite verification budget across failure classes rather than spreading it uniformly across the full output space.

3.4 Implementation

The tensor interface requires no architectural changes to current transformers. Generation APIs already support returning logits; the entropy trace is a simple transformation. Attention patterns are available with `output_attentions=True` in standard frameworks. Figure 4 shows the tensor interface.

We provide a reference implementation supporting OLMo models that demonstrates the interface is practical. The overhead is modest: storing per-token entropy and summarizing attention patterns adds approximately 2–7% latency depending on signal set, compared to generation itself.

What the tensor provides. The entropy trace provides a falsifiability signal. If a model claims certainty (“Paris is definitely the capital of France”) while its entropy trace shows high uncertainty, the mismatch is detectable. The attention summary provides complementary signal: fabrications tend to show more dispersed attention patterns than grounded responses.

What the tensor does not provide. The tensor interface is not a lie detector. It does not prove that low-entropy outputs are true or that high-entropy outputs are false. It provides *one additional signal* that, combined with other verification methods, can improve epistemic assessment. The theorems in ?? prove that *some* additional signal is necessary; the tensor interface demonstrates that such signals are available without architectural change.

4 Evaluation

The evaluation measures the cost-effectiveness of different verification strategies: what accuracy improvement does each level of verification investment buy, and how does the return on investment change as the supervisor moves from text-channel to tensor-channel signals?

4.1 Verification Cost-Effectiveness

We evaluate the return on verification investment across four strategies operating at three budget levels. The experimental design directly instantiates the conditions of our impossibility theorems: a bounded supervisor with a fixed verification budget must allocate that budget across model outputs, with the goal of maximizing end-to-end accuracy delivered to the user. The question is not merely whether tensor signals help, but how much accuracy each verification dollar buys under each strategy.

Query set. We construct a balanced set of 200 queries: 100 knowable (verifiable facts spanning geography, science, history, and biography) and 100 unknowable (fabrication prompts for nonexistent people, fictional syndromes, future events, and nonexistent citations). Ground truth labels are established by construction. The 50/50 balance maximizes statistical power for discrimination analysis; in deployment, the base rate of fabrication-inducing queries will vary by domain and the operating point must be calibrated accordingly.

Models. We run all queries on four architectures: OLMo-3 7B [9], Llama-3.1 8B [8], Qwen3 4B [11], and Mistral 7B [4], using instruct-tuned variants to ensure consistent question-answering behavior across all models.

Conditions. For each model, outputs are evaluated under four conditions sharing a fixed verification budget (the fraction of outputs the judge may verify and intervene on):

1. **No judge.** Raw model output. Establishes the unverified baseline.

Table 2. End-to-end accuracy (%) under bounded verification, averaged across four architectures (OLMo-3, Llama-3.1, Qwen3, Mistral). Ground truth established by stratified evaluation validated against human review (75/80 agreement, 93.8%).

Condition	10%	20%	30%
No judge	75.8	75.8	75.8
Text-guided (length)	79.2	82.8	87.6
Tensor-guided	81.7	86.7	90.2
Composed	81.1	87.7	91.8

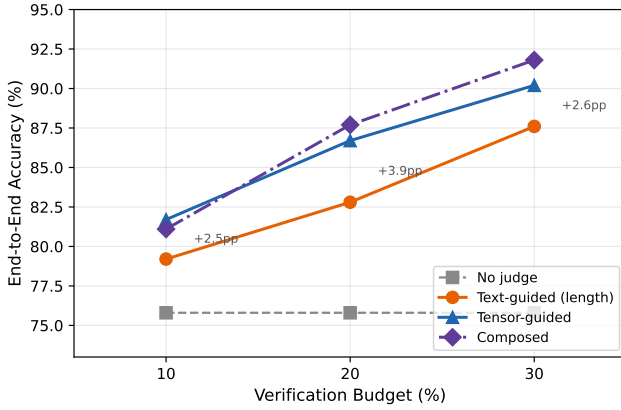


Figure 5. End-to-end accuracy vs. verification budget for four judge conditions, averaged across four architectures. The text-guided baseline uses response word count. Tensor-guided triage outperforms at every budget level by 2.5–3.9 percentage points.

2. **Text-guided judge (length).** Selects outputs for verification using response word count, a text-channel signal available without any tensor interface. Verified fabrications are replaced with abstention.
3. **Tensor-guided judge.** Same budget, same intervention strategy. Selects outputs using per-token entropy and attention summary statistics.
4. **Composed judge.** Tensor-guided selection for general queries; bounded lookup judge for citation queries where tensor signals are known to invert. Tests whether principled composition of complementary judges outperforms either alone.

Budget levels. Each condition is evaluated at 10%, 20%, and 30% verification budgets to characterize the relationship between resource investment and accuracy improvement.

Metric. End-to-end accuracy: the fraction of outputs ultimately delivered to the user that are correct, after judge intervention (verification, abstention, or replacement).

Results. Table 2 shows end-to-end accuracy across all conditions and budget levels.

Figure 5 shows that **tensor-guided verification outperforms the text baseline at every budget level**: +2.5pp at 10%, +3.9pp at 20%, +2.6pp at 30%. The text baseline uses response word count, which captures the asymmetry between

concise factual retrieval and verbose fabrication in this query set. Tensor signals discriminate among outputs of similar length, reaching cases the text channel cannot (§4.2 quantifies this advantage under length control). These numbers use probabilistic verification (success rate calibrated to human review, $p = 0.94$) rather than oracle correction; results are robust across 1,000 Monte Carlo trials.

Per-model results confirm the pattern is consistent across all four tested architectures. Tensor-guided verification improves accuracy for every model tested: OLMo (69.5% → 75.1% at 10%), Llama (82.5% → 88.1%), Qwen (87.0% → 91.7%), and Mistral (64.0% → 72.0%). Cross-model entropy agreement on the same queries yields mean pairwise Spearman $\rho = 0.762$ (rank-order correlation; $\rho = 1$ indicates identical rankings), indicating the signal is substantially driven by query-level properties rather than model-specific artifacts.

Composed judges and complementary failure modes.

At 30% budget, the composed judge (91.8%) outperforms the tensor-guided judge alone (90.2%). The advantage is small in aggregate but structurally significant: it comes entirely from citation queries where entropy signals *invert*. Fabricated citations produce lower entropy than real ones because the model generates plausible fakes more fluently than it retrieves specific bibliographic details. On citation queries, the tensor-guided judge alone underperforms; the bounded-lookup judge catches what entropy misses. This confirms the compositional defense argument: the tensor and the bounded judge have complementary failure modes, and principled composition allocates a finite budget across failure classes rather than spreading it uniformly.

Analysis: each row as a point on the cost surface. Each row maps an investment strategy to its return. Row 1 (No judge, 75.8%) is the zero-investment baseline. Row 2 (Text-guided, length) uses response word count, a text-channel signal available at zero marginal cost. Row 3 (Tensor-guided) uses per-token entropy, which discriminates among outputs of similar length, reaching cases the text channel cannot. The consistent advantage (+2.5–3.9pp across budget levels) reflects a structural difference (§4.2): text features capture behavioral patterns that vary across models; entropy captures computational uncertainty that generalizes. Row 4 (Composed) demonstrates that tiers compose: different judges cover different failure classes, achieving the highest accuracy at 20% and 30% budget levels.

Ground truth validation. Establishing reliable ground truth is itself a bounded verification problem. Our initial evaluator used substring matching and refusal markers, which is precisely the “text-only bounded supervisor” our theorems describe, and achieved only 68.8% accuracy, exhibiting the predicted failure modes (negation blindness, encoding mismatch, hedged fabrications passing as refusals). The correction required a stratified evaluator combining programmatic verification with LLM-assisted classification:

Table 3. AUC for knowable vs. unknowable discrimination under length control (long-knowable vs. long-unknowable, both 170–200 words). No text-channel feature exceeds 0.70 pooled; per-token entropy is the only signal that does. Individual text features occasionally achieve high AUC for specific models (e.g., refusal density 0.900 for Qwen) but none generalizes.

	OLMo	Llama	Qwen	Mistral	Pool
Word count	.597	.193	.362	.330	.369
Hedge density	.837	.551	.698	.435	.627
Refusal density	.605	.563	.900	.520	.648
Avg word len	.742	.514	.782	.672	.673
Avg sent len	.738	.724	.662	.598	.685
Type-token ratio	.473	.635	.570	.562	.572
Entropy (tensor)	.949	.888	.856	.642	.757

an instance of the escape condition applied to the evaluation pipeline itself. Validated against human review of 80 blinded items, agreement was 75/80 (93.8%), with five characterizable disagreements orthogonal to experimental conditions. Note: the Tier 2 LLM classifier (Qwen3 4B) is also one of the four test models; however, the evaluation task (binary classification of answer correctness) differs from the generation task, and human calibration validates the evaluator independently.

4.2 Confound Analysis: Text Features vs. Tensor Signals

The bounded verification results above show that tensor-guided triage is cost-effective. But a natural objection arises: is the tensor signal merely a proxy for text-channel features? Response length, hedging language, and vocabulary complexity are all observable without a tensor interface. If they discriminate equally well, the tensor interface adds cost without value.

Length-controlled design. Our primary query set has a systematic asymmetry: knowable queries produce concise factual answers while unknowable queries produce verbose fabrications. We construct a length-controlled evaluation with a 2×2 design: short/long $\times$ knowable/unknowable, with 25 queries per cell evaluated on all four architectures (400 total evaluations). Long-knowable queries ask for detailed explanations of verifiable topics (e.g., “Explain the CAP theorem and give an example of each tradeoff”). Long-unknowable queries prompt fabrication at similar length (e.g., “Describe the Petrov-Schwarzmann conjecture and its implications for emergent reasoning”). Both cells produce responses of 170–200 words, eliminating response length as a discriminator.

Results. Table 3 compares six text-channel features against per-token entropy in the length-controlled comparison. No text feature exceeds pooled AUC 0.70. Per-token entropy is the only signal that does.

The pattern has two components. First, text features that discriminate for one model fail for others: hedging density achieves 0.837 for OLMo but 0.435 for Mistral; refusal density achieves 0.900 for Qwen but 0.563 for Llama. Each text feature captures a behavioral idiosyncrasy of a particular model’s training, not a computational property that generalizes. Second, per-token entropy exceeds 0.85 for three of four models and achieves the highest pooled AUC of any signal tested.

Combined text classifiers do not generalize. A natural objection is that combining text features might outperform any individual feature. We trained a logistic regression on all six text features. Within-distribution (10-fold cross-validation on the length-controlled data), the combined classifier achieves pooled AUC 0.799, exceeding entropy’s 0.757. However, when trained on the primary query set (§4.1) and tested on the length-controlled data, the classifier collapses to AUC 0.511, no better than random, while entropy remains at 0.757 without any training data. The combined classifier learned distributional patterns (verbose fabrication, terse facts) that do not transfer when the distribution changes. Per-token entropy requires no labeled data and no distributional assumptions; it measures the computation directly.

This is the impossibility result instantiated empirically. Text-channel features carry behavioral signals that vary across training procedures and query distributions. A model trained to hedge less, refuse differently, or use shorter sentences defeats the corresponding text detector without changing the computation that determines whether it is fabricating. A combined classifier trained on one query distribution collapses on another. Per-token entropy captures a computational property, the model’s distributional uncertainty over next tokens, that the model cannot independently control under standard training objectives, the same structural pattern that makes code plagiarism detection (MOSS) robust [1, 12].

Negative result: entropy vs. correctness. When we test whether entropy predicts *correctness* (using TruthfulQA [6] correct vs. incorrect answer pairs), AUC drops to approximately 0.53, which is no better than random. This distinction is central to our claim: the tensor interface escapes the *verification* impossibility, not the *correctness* problem. Entropy discriminates *epistemic mode*, whether the model is retrieving or fabricating, not whether a retrieved answer is right. These are different tasks with different difficulty: mode detection requires distinguishing computational regimes; correctness detection requires ground-truth access that no internal signal can provide. Triage operates on the former: it identifies which outputs to spend verification budget on, not which outputs are correct. The budget curve (Table 2) demonstrates that entropy-guided triage allocates verification resources more effectively than text-guided triage, even though neither signal predicts correctness directly.

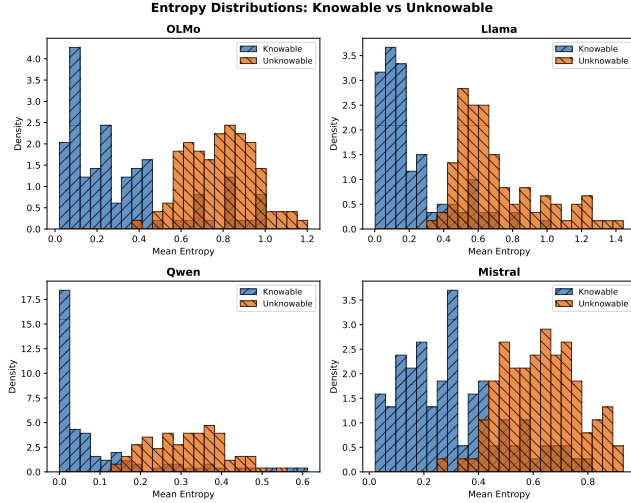


Figure 6. Distribution of mean per-token entropy for knowable vs. unknowable queries across four model architectures. The distributions are separable in every model tested, confirming that entropy discriminates knowable from unknowable queries across all tested architectures.

Self-reported confidence. Self-reported confidence is inverted across all four architectures (AUC 0.28–0.36, all below random; see §2.3).

5 Conclusion

Systems incorporating language model components must decide how much resources to spend on verification and where to spend it. We introduce the notion of epistemic observability to provide the cost surface for making that decision. We present different tiers of epistemic observability for providing different levels of verification, and build a tensor interface at the logit-based observability tier to show how it can reduce the budget required for reducing hallucinations and improving accuracy.

References

- [1] Alex Aiken. 2000. Moss: A System for Detecting Software Plagiarism. <https://theory.stanford.edu/~aiken/moss/>. Accessed: 2026-02-14.
- [2] Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Jackson Kernion, Kamal Ndousse, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, and Jared Kaplan. 2021. A General Language Assistant as a Laboratory for Alignment. arXiv:2112.00861 [cs.CL] <https://arxiv.org/abs/2112.00861>
- [3] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. 2023. Discovering Latent Knowledge in Language Models Without Supervision. In *International Conference on Learning Representations (ICLR)*.

- <https://openreview.net/forum?id=ETKGuby0hcs>
- [4] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] <https://arxiv.org/abs/2310.06825>
 - [5] Konstantin Krestnikov. 2026. Truth as a Compression Artifact in Language Model Training. arXiv:2603.11749 [cs.CL] <https://arxiv.org/abs/2603.11749>
 - [6] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 3214–3252.
 - [7] Christina Lu, Jack Gallagher, Jonathan Michala, Kyle Fish, and Jack Lindsey. 2026. The Assistant Axis: Situating and Stabilizing the Default Persona of Language Models. arXiv:2601.10387 [cs.CL] <https://arxiv.org/abs/2601.10387>
 - [8] Meta AI. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] <https://arxiv.org/abs/2407.21783>
 - [9] Team Olmo, :, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, Jacob Morrison, Jake Poznanski, Kyle Lo, Luca Soldaini, Matt Jordan, Mayee Chen, Michael Noukhovitch, Nathan Lambert, Pete Walsh, Pradeep Dasigi, Robert Berry, Saumya Malik, Saurabh Shah, Scott Geng, Shane Arora, Shashank Gupta, Taira Anderson, Teng Xiao, Tyler Murray, Tyler Romero, Victoria Graf, Akari Asai, Akshita Bhagia, Alexander Wettig, Alisa Liu, Aman Rangapur, Chloe Anastasiades, Costa Huang, Dustin Schwenk, Harsh Trivedi, Ian Magnusson, Jaron Lochner, Jiacheng Liu, Lester James V. Miranda, Maarten Sap, Malia Morgan, Michael Schmitz, Michal Guerquin, Michael Wilson, Regan Huff, Ronan Le Bras, Rui Xin, Rulin Shao, Sam Skjonsberg, Shannon Zejiang Shen, Shuyue Stella Li, Tucker Wilde, Valentina Pyatkin, Will Merrill, Yapei Chang, Yuling Gu, Zhiyuan Zeng, Ashish Sabharwal, Luke Zettlemoyer, Pang Wei Koh, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2025. Olmo 3. arXiv:2512.13961 [cs.CL] <https://arxiv.org/abs/2512.13961>
 - [10] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. arXiv:2203.02155 [cs.CL]
 - [11] Qwen Team. 2025. Qwen3 Technical Report. Alibaba Cloud technical report. <https://qwenlm.github.io/blog/qwen3/>
 - [12] Saul Schleimer, Daniel S. Wilkerson, and Alex Aiken. 2003. Winnowing: Local Algorithms for Document Fingerprinting. In *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*. ACM, 76–85. doi:10.1145/872757.872770
 - [13] Zhepei Wei, Xiao Yang, Kai Sun, Jiaqi Wang, Rulin Shao, Sean Chen, Mohammad Kachuee, Teja Gollapudi, Tony Liao, Nicolas Scheffer, Rakesh Wanga, Anuj Kumar, Yu Meng, Wen tau Yih, and Xin Luna Dong. 2025. TruthRL: Incentivizing Truthful LLMs via Reinforcement Learning. arXiv:2509.25760 [cs.CL] <https://arxiv.org/abs/2509.25760>
 - [14] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. 2024. Hallucination is Inevitable: An Innate Limitation of Large Language Models. arXiv:2401.11817 [cs.CL]